Made With Gsap

045
Folders

In this effect, we will arrange a set of slides one behind the other with a perspective. As the user scrolls, the slides will scroll infinitely, and the closest slide will disappear and be placed at the back of the queue. Are your wrists warmed up? Then let's get started.

HTML Structure

The root division will undergo a slight rotation based on the position of the cursor. Its children will be the various slides to scroll.

```
<section class="mwg_effect045">
    <div class="root">
        <div class="slide">...</div>
        <div class="slide">...</div>
        <div class="slide">...</div>
        ...
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Some CSS

Let's now set up the 3D view with CSS. First, we need to define a perspective:

```
.mwg_effect045 {
    perspective: 150vw;
}
```

I like to specify the perspective in `vw` to maintain the same effect regardless of screen width. The smaller the value, the more pronounced and deeper the perspective will be. To preserve this perspective on the children, I used `transform-style: preserve-3d` on our `root` division.

```
.mwg_effect045 .root {
    transform-style: preserve-3d;
}
```

Finally, each `slide` will not be perfectly centered on the parent's height to account for the entire queue.

```
.mwg_effect045 .slide {
    ...
    top: calc(50% - 14vw); /* not exactly half to center with perspective */
}
```

Beginning of the timeline

The animation of each `slide` will be contained within a `gsap.timeline()`. All the tweens will be linked to the same timeline.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

For the script to work regardless of the number of slides in the DOM, I start by choosing a reference duration based on that number.

```
const slides = document.querySelectorAll('.mwg_effect045 .slide')
const baseDuration = slides.length / 2 // total duration for the slide animatio
```

My first tween in the timeline uses the `gsap.from()` method. This method animates an element from a specified initial state to its current state. Here, my slides will start from the back of the screen, be off-center on the y-axis, and then move to the front and center. In the `stagger` object, we can notice the `each: 0.5` property. The `baseDuration` variable divided by the number of items equals 0.5 as well. Thus, each slide will move evenly.

```
const staggerEach = 0.5; // Delay between each element

tl.from(slides, {
    y: '-15vw',
    z: '-60vw',
    ease: "none", // Linear movement
    duration: baseDuration,
    stagger: {
        each: staggerEach, // baseDuration / slides.length = 0.5
        repeat: -1 // Infinite repeat
    }
})
```

Appaerance tween

This tween will handle the movement of the slide that reappears at the back of the queue. It uses the `fromTo()` method, which lets you define both the starting and ending values for an animation. I specify that it will play every total duration minus its own `duration`, thanks to the `repeatDelay` variable.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

```
:ect045 .content')
```

```
// Content entry animation
tl.fromTo(slideContent, {
    y: '10vh' // Start slightly off-center on the y-axis
}, {
    y: 0, // Quickly return to 0 to center it in its parent slide
    ease: "back.out(1.05)",
    duration: staggerEach, // 0.5
    stagger: {
        each: staggerEach, // 0.5
        repeat: -1, // Infinite repeat
        repeatDelay: repeatDelay, // baseDuration - staggerEach
        onRepeat() {
            // Target the repeated slide and move it off-screen
            this.targets()[0].style.transform = "translateY(100vh)";
        }
    }
}, '<') // Means the animation starts at the start of the previous tween
```

Disappearance tween

When the slide reaches the front of the queue, I make it quickly move below the viewport. I keep the same structure as the appearance animation, but since I want the animation to play just before the end of my `timeline()` , I specify `delay: repeatDelay` here.

```
// Content exit animation
tl.fromTo(slideContent, {
    y: 0, // Start at the center on the y-axis
}, {
    y: '200vh', // Quickly move below the viewport
    ease: "power3.in",
    duration: staggerEach, // 0.5
    delay: repeatDelay, // baseDuration - staggerEach
    stagger: {
        each: staggerEach, // 0.5
        repeat: -1, // infinite repeat
        repeatDelay: repeatDelay, // baseDuration - staggerEach
        onRepeat() {
            // Target the repeated slide and center it in its parent
            this.targets()[0].style.transform = "translateY(0vh)";
        }
    }
}, '<') // Means the animation starts at the start of the previous tween
```

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

ached to a CSS property of an ached to an object. This method allows me to have a progression between two new values. With each change, the progression of the timeline will correspond to the value in `tl.time()` .

To avoid any display glitches before the user starts scrolling, I force an initial movement in my timeline.

```
const deltaObject = {delta: 0}

deltaTo = gsap.quickTo(deltaObject, 'delta', { // Tied to delta from deltaObject
    duration: 0.8, // Will take 0.8 sec to reach its new value
    ease: "power1", // Non linear
    onUpdate: () => {
        tl.time(deltaObject.delta) // As soon as it changes, we progress the timeline
    }
})
```

To avoid any display glitches before the user starts scrolling, I force an initial movement in my timeline.

```
const beginDistance = slides.length * 100 // Initial High Progression

tl.time(beginDistance)
// animates from beginDistance + 0.01 to beginDistance
deltaTo(beginDistance + 0.01, beginDistance)
```

I then declare a `wheel` event that will be called as soon as the user scrolls. In this event, I increment a variable to navigate between my slides in both directions.

```
let incr = 0

window.addEventListener("wheel", (e) => {
    incr += e.deltaY / 1000 // divided per 1000 to slow it down
    deltaTo(incr + beginDistance) // variation de deltaObject.delta
}, {passive: true})
```

We can now navigate between the slides by scrolling. For a more solid result, I want the movement to stop only when a slide is perfectly in front of the screen. To do this, I use a

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

hod allows you to snap to a value in an array.

```
let incr = 0

// establish the value to snap to a certain increment
// here for each item, so baseDuration / slides.length
const snap = gsap.utils.snap(baseDuration / slides.length)

window.addEventListener("wheel", (e) => {
    incr += e.deltaY / 1000 // Divided per 1000 to slow it down
    // Change deltaObject.delta, takes the snapped value as a parameter
    deltaTo(snap(incr + beginDistance))
}, {passive: true})
```

Add more effect

To go further, I want to modify the displayed perspective based on mouse movements: I declare two new `gsap.quickTo()` methods that will be assigned to the `rotation` properties of my root division. These methods help avoid calling multiple `gsap.to()`, which would hurt performance.

```
const rotY = gsap.quickTo(root, "rotationY", {duration: 0.3, ease: 'power3'})
const rotX = gsap.quickTo(root, "rotationX", {duration: 0.3, ease: 'power3'})
```

Now, I just need to provide `rotY` and `rotX` with values based on the position of my cursor. So, I need to declare a `mousemove event`.

```
root.addEventListener("mousemove", e => {
    const valX = (e.clientY / window.innerHeight - 0.5) * 5
    const valY = (e.clientX / window.innerWidth - 0.5) * 10

    rotX(-valX)
    rotY(valY)
})
```

Here, `e.clientX / window.innerWidth` varies between 0 and 1. By subtracting 0.5, the expression now varies between -0.5 and 0.5. Then, I simply multiply by a number to increase the angle.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

could be interesting to lengthen the `duration` of the appearance tween and the disappearance tween. This way, the slide would start disappearing well before it reaches the front. We could also modify the different `ease` and, for example, add a bounce before disappearing using `elastic.in()`.

Final code

HTML CSS JAVASCRIPT

```html
<section class="mwg_effect045">
    <div class="text">
        <p>Recent works</p>
        <p>2023-2025</p>
    </div>
    <div class="root">
        <div class="slide">
            <img class="content" src="assets/medias/1.png" alt="">
        </div>
        <div class="slide">
            <img class="content" src="assets/medias/2.png" alt="">
        </div>
        <div class="slide">
            <img class="content" src="assets/medias/3.png" alt="">
        </div>
        <div class="slide">
            <img class="content" src="assets/medias/4.png" alt="">
        </div>
        <div class="slide">
            <img class="content" src="assets/medias/5.png" alt="">
        </div>
        <div class="slide">
            <img class="content" src="assets/medias/6.png" alt="">
        </div>
        <div class="slide">
            <img class="content" src="assets/medias/7.png" alt="">
        </div>
    </div>
</section>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Wanna learn something else?

BACK TO THE COLLECTION

Never miss what's next

Your email SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap PRIVACY POLICY